



CST4090 Individual Project

Title: Explainable AI-Based Resume Scoring System

Student Name: Keya Barua

Student ID: M01016132

Supervisor Name: David Windridge

ABSTRACT

Artificial Intelligence is being widely used in the recruitment process to screen thousands of resumes which reduced the workload of the hiring managers but these systems are black-box models. These do not provide any explanations for the decisions provided. This leads to mistrust, confusion and disappointment among the job seekers and the recruiters. Additionally, the candidates do not understand the reason behind their CV not getting shortlisted for a particular job because the current systems provide an analysis but not interpretation. This project “Explainable AI-Based Resume Scoring System” is an interpretable, candidate-facing resume-JD scorer that returns a calibrated match decision, a 0-100 score, and transparent reasons. Rather than relying on opaque deep encoders, the system uses TF-IDF features built on the concatenated JD-Resume pair and a Logistic Regression classifier whose raw scores are calibrated (Isotonic / Platt). A gold evaluation set from the Adzuna job adverts by pairing each JD with its positive resume(s) and sampled non-matches, splitting by JD to avoid leakage. On this benchmark, the FUSED model consistently outperforms the baseline models, TF-IDF cosine, Sentence-BERT bi-encoder cosine and a weak re-ranker across the ranking metrics. The metrics used were MRR@5, Recall@k and NDCG@k, where $k \in \{5, 10, 50\}$. The FUSED model metrics are MRR@5: 0.248, Recall@5: 0.39, Recall@10: 0.59, NDCG@5: 0.279, NDCG@10: 0.349. The metrics for the TF-IDF cosine model are MRR@5: 0.204, Recall@5: 0.364, Recall@10: 0.567, NDCG@5: 0.241, NDCG@10: 0.30. The Bi-encoder underperformed with MRR@5: 0.143, Recall@5: 0.297 and Recall@10: 0.418. The probability calibration improves reliability curves towards the identity line. Stress checks indicate reasonable robustness. Surrogate fidelity to the linear explanation is high, counterfactual token injection produces tiny shifts, and performance remains stable across JD categories. The headline score in the Streamlit application is a fused signal that blends calibrated probability with TF-IDF cosine similarity and JD-term overlap, and an operating threshold chosen from the validation Precision-Recall curve supports two modes (high-recall screening vs high-precision shortlisting). Explanations are term level and bar charts show positive and negative evidences. A waterfall plot traces baseline to prediction contributions, and a JD presence view highlights important JD terms that are missing from the resume, yielding actionable edits. The result is a lightweight, fast, and auditable screening tool that converts AI scoring into a learning loop for applicants, offering concrete guidance on what to emphasize, rephrase, or add, thereby promoting transparency and fairness in early-stage hiring.

Content page

Abstract.....	i
1. Introduction.....	1
2. Background & Literature Review.....	3
2.1. Black-box Models and XAI Approaches.....	3
2.2. Fairness & Bias in AI Recruitment.....	4
3. Methods.....	5
3.1. Data and Preprocessing.....	5
3.2. Exploratory Data Analysis.....	7
3.3. Text Representation.....	10
3.4. Resume-JD Scoring & Calibration.....	10
3.5. Explainability and Feedback Generation.....	11
3.6. Model Selection.....	11
3.7. System Integration.....	12
4. Results and Evaluation.....	12
4.1. Experimental Setup & Baselines.....	12
4.2. Evaluation.....	13
4.3. Explainability, Fidelity & Robustness.....	16
4.4. App UI & Explainability.....	18
4.5. Error Analysis.....	21
5. Discussion and limitations.....	21
6. Conclusion	23
7. References.....	25

1. INTRODUCTION

Artificial intelligence (AI) has become an integral component of modern recruitment processes. Organizations are increasingly adopting automated systems to evaluate candidates. Though these technologies have significantly reduced the workload of human resource (HR) managers by enabling the rapid screening of thousands of resumes, several concerns remain regarding the transparency, fairness, and accountability of such systems. Most existing AI-driven resume screening tools operate as “black-box” models, where users can observe the input (the resume) and the output (the result or ranking) only. They are not able to discern how the system arrives at the decision. As Kosinski (2024) notes, this opacity raises questions of trust for both applicants and recruiters, as it remains unclear which features contributed and how they were weighted in the decision-making process.

The lack of interpretability not only undermines confidence in AI-based recruitment but also risks perpetuating or amplifying existing biases. For example, systems trained on historical recruitment data may inadvertently encode gender, racial, or socio-economic biases, leading to unfair or discriminatory outcomes (Mijtaba & Mahapatra, 2024; Fabris et al., 2023). Empirical evidence confirms these risks. For example, Amazon abandoned its AI recruitment tool after it was found to systematically penalize resumes containing the word “*women’s*”, such as “women’s chess club captain”, because the model had been trained on data reflecting male-dominated tech hiring dataset (Dastin, 2018). Similarly, LinkedIn’s targeted advertising systems were observed to disproportionately show higher-paying technical roles to men while displaying administrative positions more often to women, reinforcing occupational stereotypes (Lambrecht & Tucker, 2019).

Due to the growing reliance on AI in HR management, it has led to widespread mistrust in automated recruitment platforms (Blessing, 2025) and these issues highlight the urgent need for transparent and explainable solutions. Controlled studies have also demonstrated discriminatory outcomes. A study at University of Washington showed that resumes with white-associated names were preferred 85% of the time compared with only 9% for resumes with Black-associated names, while male-associated names were also more frequently selected than female-associated names despite equivalent qualifications (University of Washington, 2024). Additionally, AI hiring tools have been reported to disadvantage Black male applicants even when their credentials matched those of other demographic groups (VoxDev, 2024). These examples demonstrate that fairness in recruitment AI cannot be assumed and that bias mitigation is essential.

Beyond model opacity and bias issues, there is a practical feedback gap for job seekers. Most screening pipelines return no specific rationale, leaving candidates unsure what to add, rephrase or emphasize to align with a job description. Access to high-quality resume review is uneven, mentor feedback is scarce and performing A/B “experiments” on applications is

infeasible. This results in high variance in outcomes driven by phrasing and emphasis rather than underlying competence which is an effect amplified by black-box rankers.

The project presented “**Explainable-AI Resume Scoring System**” addresses these gaps and issues by developing an Explainable AI (XAI)-based resume scoring system, a candidate-facing tool which, unlike the conventional black-box models, seeks to evaluate resumes not only for relevance to job descriptions (JD) but also with interpretable feedback that reveals the underlying rationale behind the generated scores. It not only scores alignment to a job description but also shows “why” the resume was provided this rank. It highlights the terms that helped or hurt and which important JD elements are missing. The feedback is actionable: candidates learn which skills or experiences to foreground, where wording can be aligned to the JD, and what to add or clarify to strengthen their profile. The goal is a fairer, more trustworthy screening experience that turns a one-way filter into a learning loop for applicants.

The primary goal of this project is to design and implement a functional prototype that evaluates resumes against JDs using machine learning (ML) models enhanced with explainability techniques. The methodology involves collecting and preprocessing datasets of resumes and job descriptions, training classification models with calibrated probabilities, and integrating explanation mechanisms to highlight the most influential textual features. Techniques such as token-level contribution analysis have been implemented within this prototype, with extensions using SHAP to provide richer interpretability. To address fairness, preprocessing steps and feature selection strategies are employed to mitigate the influence of demographic or irrelevant features, thereby reducing bias in decision-making (Magham, 2024).

Given a JD and a resume, the system returns a decision “Pass” or “Review” and a 0-100 match score. Additionally, it shows “why”, which terms helped or hurt and which important JD terms are missing so that candidates know what to add, rephrase, or foreground. Under the hood, it uses TF-IDF over the concatenated JD and Resume pairs, a Logistic Regression with isotonic / Platt calibration and a fused score with a fixed threshold. Explanations come from exact term contributions, a JD-presence view, and a waterfall path from baseline to prediction. Fairness is supported via deterministic preprocessing, including PII avoidance and calibration or reliability checks so probabilities align with observed outcomes.

In summary, the rising adoption of AI in recruitment has improved efficiency but introduced significant challenges of transparency, fairness, and accountability. Existing black-box models such as BERT and GPT provide strong predictive performance but lack interpretability, which undermines trust among hirers, HR managers, and job seekers. This project addresses these issues by developing Explainable AI-based Resume Scoring System that combines ML with interpretability techniques and bias mitigation strategies. It focuses on candidate empowerment. It keeps the scoring pipeline auditable, presents clear reasons behind each result, and offers targeted, JD-specific guidance to help applicants revise

effectively. By combining calibrated modeling with transparent explanations, the system aims to make AI-assisted screening more equitable, understandable, and useful for job seekers.

2.BACKGROUND & LITERATURE REVIEW

Artificial Intelligence (AI) has become increasingly prominent in recruitment, where the automation of resume screening is a common practice now. Organizations deploy ML models and **Natural Language Processing (NLP)** techniques to assess candidate suitability against job descriptions, enabling faster and more consistent decision-making compared with manual review (Blessing, 2025). The rising adoption of AI-driven recruitment systems reflects their ability to manage large volumes of applications and identify candidates with relevant skills and experience more efficiently than traditional processes. However, this efficiency is accompanied by concerns about transparency, fairness, and accountability, which form the core focus of the dissertation project.

2.1. Black-box Models and XAI Approaches

Recent systems leverage advanced NLP models, such as BERT and GPT, to evaluate semantic similarity between resumes and JDs. These models are able to understand contextual relationships in texts beyond simple keyword matching, thereby improving accuracy in identifying candidate relevance. A perfect example would be LinkedIn Talent Solutions, which employs BERT-based models to perform semantic analysis of resumes and job postings, improving the ranking of candidates for recruiters (LinkedIn, n.d.). Similarly, Google's Cloud Talent Solution API incorporates semantic search and ML ranking algorithms to match candidates with job descriptions, further demonstrating the integration of advanced NLP into recruitment workflows (Google Cloud, 2025). However, despite strong performance, such systems lack interpretability. The internal mechanisms of the deep learning models like BERT are opaque, making it difficult for hirers or candidates to understand how the decisions are made.

Explainable AI (XAI) has emerged as a response to this opacity, with methods such as **SHAP (Shapley Additive Explanations)** and **LIME (Local Interpretable Model-Agnostic Explanations)** providing human-interpretable explanations for model outputs. These approaches generate feature attributions that reveal the influence of input variables on predictions, offering transparency to users (Fabeyo et al., 2025). For instance, SHAP has been applied to highlight which specific words or phrases in a resume most influenced a model's scoring decision, allowing both job seekers and hirers to see the rationale behind a recommendation or ranking. As Hofeditz et al. (2022) demonstrated, applying XAI techniques to candidate recommendation systems enables the recruiters of an organization to better understand the reason behind the recommendation of certain applicants, improving trust in technology. Similarly, Magham (2024) highlights the potential of XAI to mitigate bias by exposing hidden patterns in decision-making processes that may otherwise go unnoticed, such as the overemphasis on education level or employment gaps.

Besides recruitment, XAI techniques have also shown value in other high-stakes decision-making domains. In healthcare, SHAP has been successfully applied to clinical prediction models to explain patient risk factors, enabling physicians to validate AI outputs before acting on them (Lundberg et al., 2018). In finance, LIME has been used to explain credit-scoring decisions, allowing regulators and customers to understand why certain loan applications were approved or disapproved (Ribeiro et al., 2016). These examples illustrate the versatility of XAI approaches and highlight their potential to enhance trust and accountability in recruitment systems where similar issues of transparency and fairness are at stake.

2.2. Fairness and Bias in AI Recruitment

Bias and fairness remain significant challenges in AI recruitment systems. Historical datasets which are used for model training often contain imbalances reflecting gender, racial, or socio-economic disparities, which can be amplified by machine learning (ML) algorithms (Fabris et al., 2023). Mujtaba and Mahapatra (2024) categorized fairness metrics and mitigation strategies, emphasizing the need for pre-processing approaches such as rebalancing datasets, feature filtering to remove sensitive attributes, and post-processing techniques to adjust biased outputs. Similarly, Oni (2025) underscores the ethical and regulatory imperatives for transparency in recruitment systems, pointing out that XAI plays a central role in ensuring compliance with fairness standards. However, it is important to note that explainability alone does not guarantee fairness. Cecil et al. (2024) show that even models equipped with interpretability mechanisms can still perpetuate unfair outcomes if the underlying training data is biased.

Research has shown that bias can manifest in subtle yet significant ways. For example, Barocas and Selbst (2016) demonstrated how predictive algorithms may reproduce structural inequalities when trained on historical employment or educational data. Raghavan et al. (2020) found that resume screening systems often rely on proxies such as postal codes or university names, which can indirectly encode socio-economic or racial disparities. Similarly, Mehrabi et al. (2021) highlighted that even when sensitive attributes such as gender or race are excluded from training, models can infer these characteristics from correlated features, perpetuating bias through indirect pathways.

Datasets also play a critical role in shaping recruitment models. The **Resume Dataset** provided by **InferencePrince555 (2021) on Hugging Face**, which consists of approximately 32,000 resumes, is a valuable resource for developing and testing automated scoring systems. In addition, access to real-world job descriptions through APIs such as Adzuna (2025) allows for realistic evaluation of the performance of the model against current market requirements. While these sources offer significant opportunities, they also present challenges in ensuring data quality and representativeness, both of which directly affect model fairness and generalizability. If datasets over-represent certain professions, demographics, or geographies, recruitment models risk producing outcomes that favor some groups while disadvantaging others (Raghavan et al., 2020; Mehrabi et al., 2021)

Beyond recruitment, studies in adjacent domains highlight the risks of unchecked algorithmic bias. For instance, Buolamwini and Gebru (2018) uncovered significant racial and gender disparities in commercial facial recognition systems, where darker-skinned women were misclassified up to 34% of the time compared with less than 1% for lighter-skinned men. Although this research did not focus on hiring directly, it illustrates how biased datasets and insufficiently tested AI systems can have profound discriminatory impacts when deployed in high-stakes decision-making.

These findings underscore the importance of fairness-aware design in recruitment AI. Bias mitigation requires not only technical interventions, such as dataset rebalancing and algorithmic auditing, but also organizational and regulatory oversight to ensure that recruitment systems promote equity rather than entrench inequality.

3. METHODS

The objective of the project was to develop a system which would take a resume or resumes and a job description and generate two complementary outputs:

- (i) a **candidate-facing view**, where a resume uploaded by a user is scored against a given JD and accompanied by interpretable explanations, and
- (ii) a **hirer-facing view**, where multiple resumes can be compared and ranked relative to the same JD.

This section covers the methodological pipeline, including data preparation, text representations, and retrieval and pairwise scoring, calibration and thresholding, explainability, model selection, and the integration of trained artifacts in the application.

3.1. Data and Preprocessing

3.1.1. Training Corpus

For training the models, the Resume Dataset from **Hugging Face (InferencePrince555, 2021)** was used. The dataset contains around 32,000 resumes and it was downloaded in CSV format for easy integration into the system. This dataset was used for unsupervised text modelling like fitting token statistics and enrich feature space.

Several preprocessing techniques were used and all the documents were normalized and cleaned in a single, deterministic pipeline. The goal of this stage was two-fold:

- (i) remove any formatting noise and personally identifiable information (PII) so the model learns on content rather than identity, and
- (ii) produce a compact, machine-readable table.

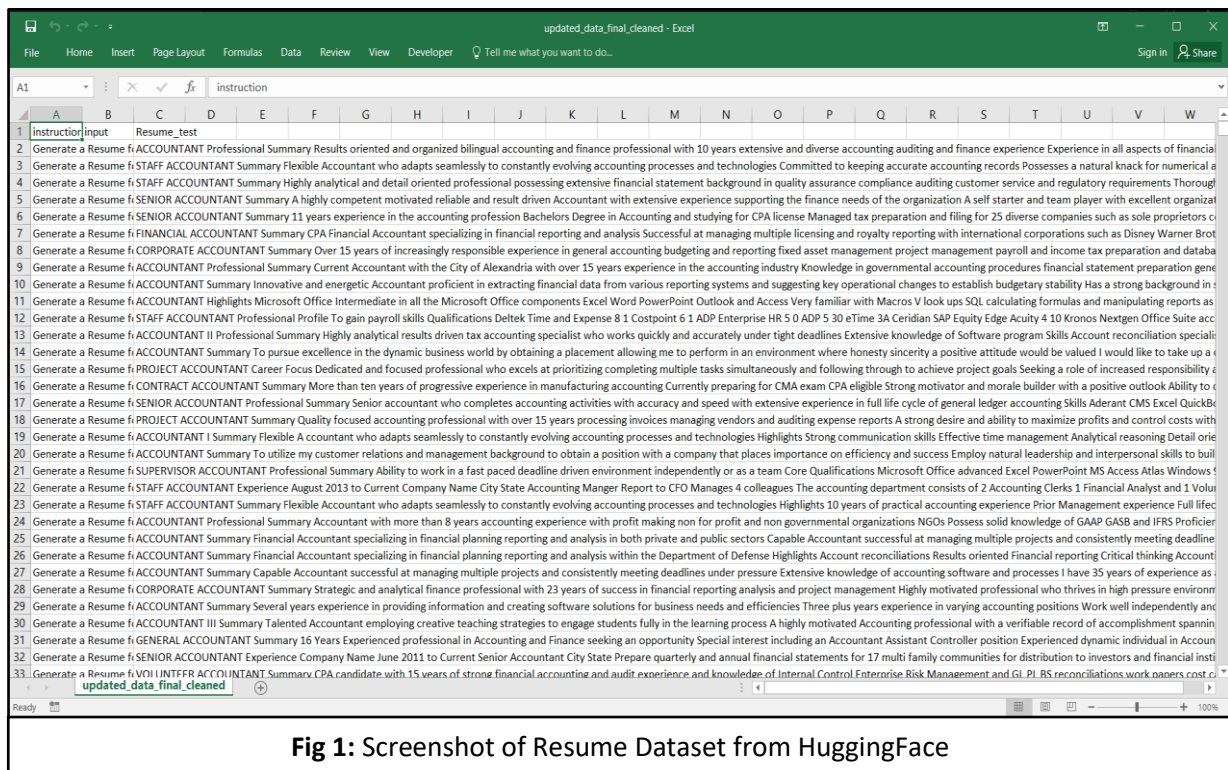


Fig 1: Screenshot of Resume Dataset from HuggingFace

The **preprocessing** of the dataset was performed in several steps. The input text was first decoded and canonicalised to NFC Unicode. Line endings were **normalized** and repeated whitespaces were collapsed to a single space so that enumerations such as bullet points and wrapped lines behaved consistently during **tokenization**. Residual markup like HTML tags, markdown bullets, page headers/footers introduced by OCR were stripped with lightweight **regex rules**. Text is lower-cased to reduce vocabulary sparsity, and common control characters are removed. This normalization made the vector space stable across different data sources (regular “Résumé”, “RESUME”, and “Resume” all map to the same token stream).

The most common PII fields were **redacted**. Email addresses, phone numbers, and URLs were detected with conservative regular expressions and replaced by sentinel tokens [EMAIL], [PHONE], and [URL]. Redaction was performed before any further tokenisation so those spans do not leak into n-grams or technology counts. After normalization, a light **segmentation** pass was performed to flatten bullet characters and preserve section headings such as “experience”, “projects”, “skills” and education”.

The intent was to keep the informative sections contiguous while removing obvious boilerplate. The cleaned text was then tokenized with a simple whitespace tokenizer so that basic statistics can be computed. Two filters were applied guard against junk inputs and empty extractions: **documents must contain at least 20 tokens and at least 100 characters**. These thresholds were chosen empirically after an initial exploratory analysis: they remove stubs and corrupted files while retaining essentially all legitimate CVs.

Duplicates were controlled in two passes. For exact duplicates, an MD5 hash over a normalized version of the text (lower-cased, punctuation collapsed) was computed and the

first instance of each hash was kept, whereas near-duplicates, which often arise from trivial formatting edits, were reduced by applying **cosine similarity** over **TF-IDF representations** and removing resumes above a similarity threshold.

3.1.2. Job-Description Corpus (Adzuna) and Gold Pairs

In addition to the resume corpus, a collection of job descriptions (JDs) was obtained using the **Adzuna API (2025)** to provide realistic and up-to-date recruitment data. Real-world JDs were included to ensure that the system evaluates resumes against current market requirements and domain-specific expectations, thereby improving its practical relevance for job seekers.

The Adzuna API was selected for its extensive coverage of job markets across multiple industries and regions. Queries were designed to retrieve job postings from diverse categories such as **Engineering, Information Technology, Education, Finance, Project Management, and Healthcare** to ensure balanced domain representation. Each query returned structured job listings containing fields including “title”, “category”, “description”, and “requirements”.

Raw job descriptions obtained from the API were preprocessed through a consistent cleaning pipeline to ensure compatibility with the resume dataset. HTML tags, markdown formatting, and other non-informative text such as company boilerplate and legal disclaimers were removed using regular expressions. The text was lowercased and normalized to NFC Unicode to maintain encoding consistency. Special characters and redundant whitespace were stripped, and non-alphanumeric tokens were filtered out. This preprocessing ensured that the textual structure of JDs aligned with that of resumes, allowing meaningful comparison during semantic similarity evaluation.

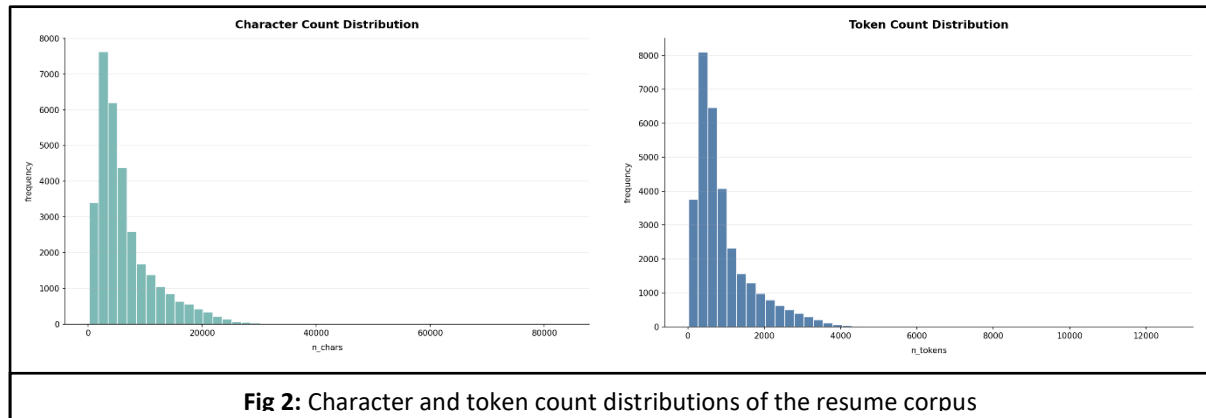
The cleaned JDs were tokenized using a whitespace tokenizer, and documents with fewer than 50 tokens were filtered out to remove incomplete or irrelevant postings. Additionally domain-level metadata (e.g., job category) was retained to facilitate exploratory analysis and ensure diversity across fields. The final JD corpus was then embedded alongside resumes to enable pairwise similarity scoring and cross-domain analysis. The gold pairs were manually labelled as Positive (match) or Negative (non-match).

The two main advantages of using live job descriptions are: first, it allowed the model to learn the linguistic patterns and key skills required in contemporary job markets and second, it improved the generalizability of the system by exposing it to multiple domains and role types. This integration of resumes and current JDs laid the foundation for developing a system that can score resumes accurately and offer feedback aligned with real hiring criteria as well.

3.2. Exploratory Data Analysis

Exploratory Data Analysis (EDA) was performed on the resume dataset to better understand the characteristics of the dataset and to inform preprocessing and model designs.

In Figure 2, the top distribution shows the character counts across resumes is right-skewed, with most resumes containing between 2,000 and 20,000 characters. This variation reflects the diversity of resume lengths across different professions. The bottom histogram presents the distribution of token counts, showing that most resumes contain between 200 and 2,000 tokens, while a small number of outliers exceed this range. These findings confirm that the chosen thresholds effectively removed incomplete or excessively long resumes.



The lexical analysis visualized in figure 3 (top unigrams and bigrams) reveals the dominance of technical and professional terminology, such as “data”, “application”, and “SQL” suggesting that the dataset is strongly skewed toward IT and software roles.

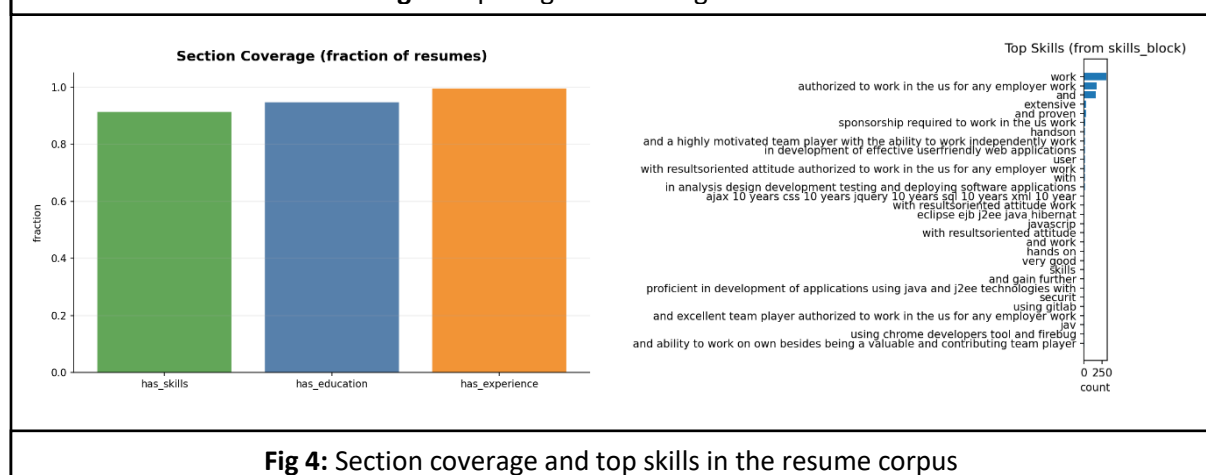
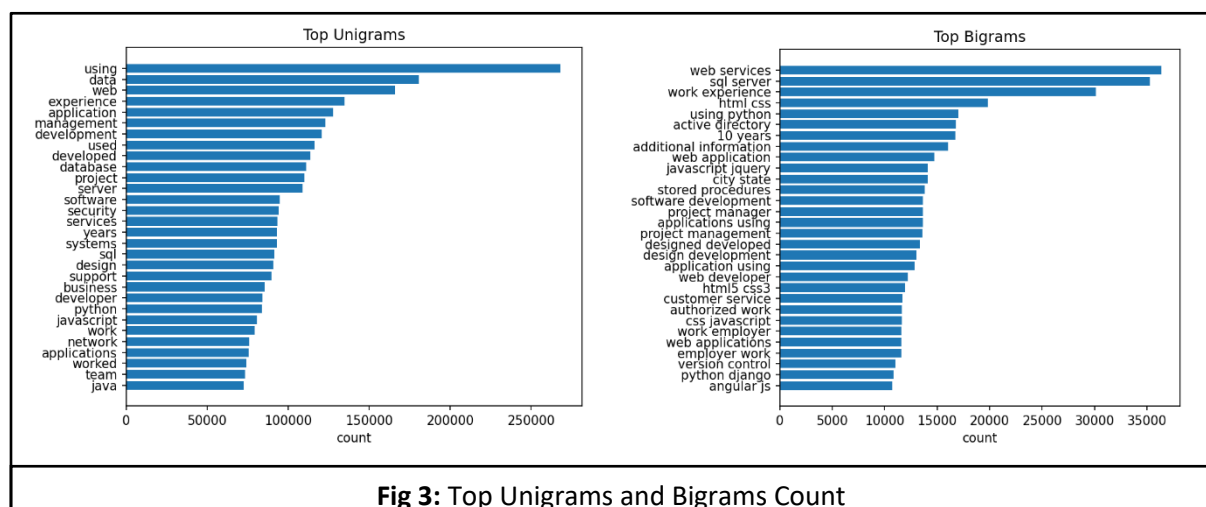
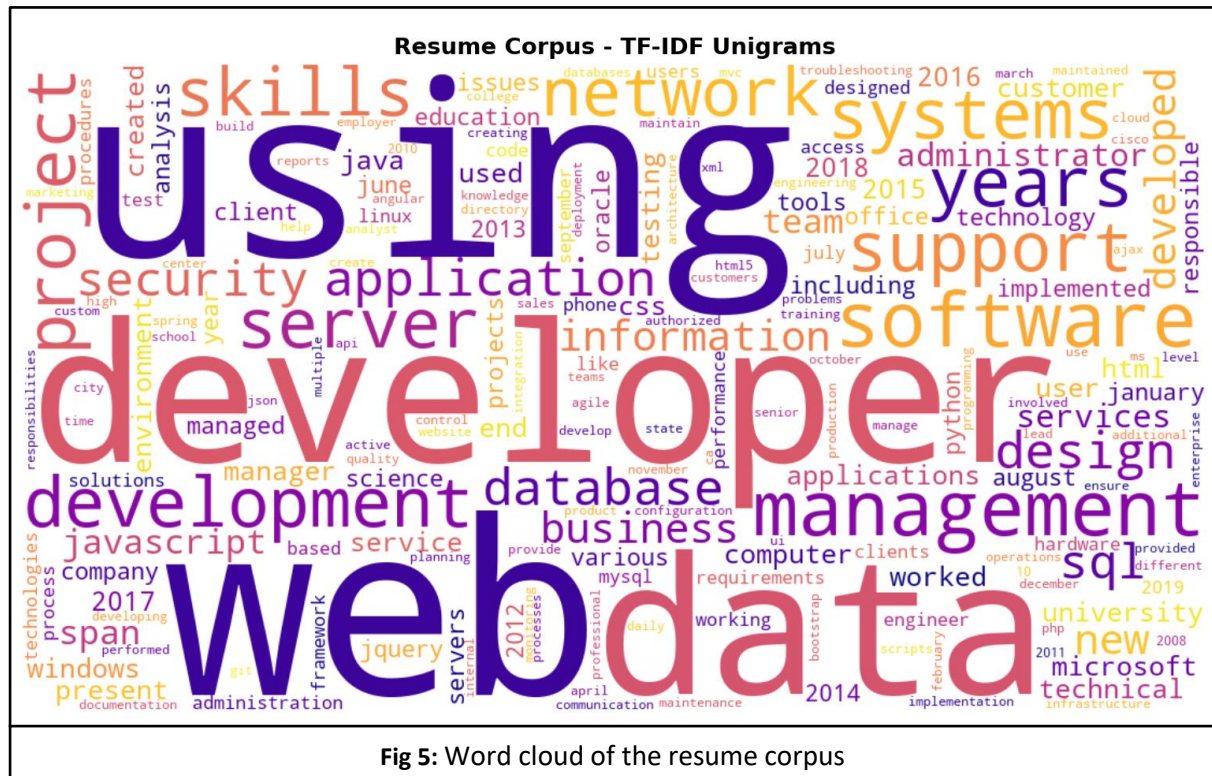


Figure 4 illustrates section coverage across resumes and top skills present in the dataset. From the left figure, it can be seen that the majority of documents contain the **Experience**, **Education**, and **Skills** sections, confirming the reliability of section-based feature extraction. The figure on the right side presents the most frequently extracted skills, showing prevalence of terms like Java, Python, and Machine Learning, alongside some noisy entries like “authorized to work in the US”. From this figure, it can be understood that while the dataset captures a wide range of competencies, additional filtering is needed to remove non-skill tokens.



To better understand the lexical composition of the resume corpus, a TF-IDF word cloud was generated, as shown in Figure 5. The visualization highlights the most frequent and influential unigrams across the dataset. Terms such as “developer”, “web”, “data” “software” and “project” dominate the corpus, indicating high concentration of resumes from technology and IT-related domains. Other frequent tokens such as “management”, “security” and “database” suggest a broad representation of technical and managerial roles.

Analysis of job descriptions also provided useful insights. As shown in figure 6 (left figure), domains such as Engineering, Hospitality, and Education are the most represented, while fields like Logistics and DevOps appear less frequently. The right figure shows that most JDs contain fewer than 100 tokens, emphasizing the need for semantic embeddings that can identify meaningful relationships in short text passages.

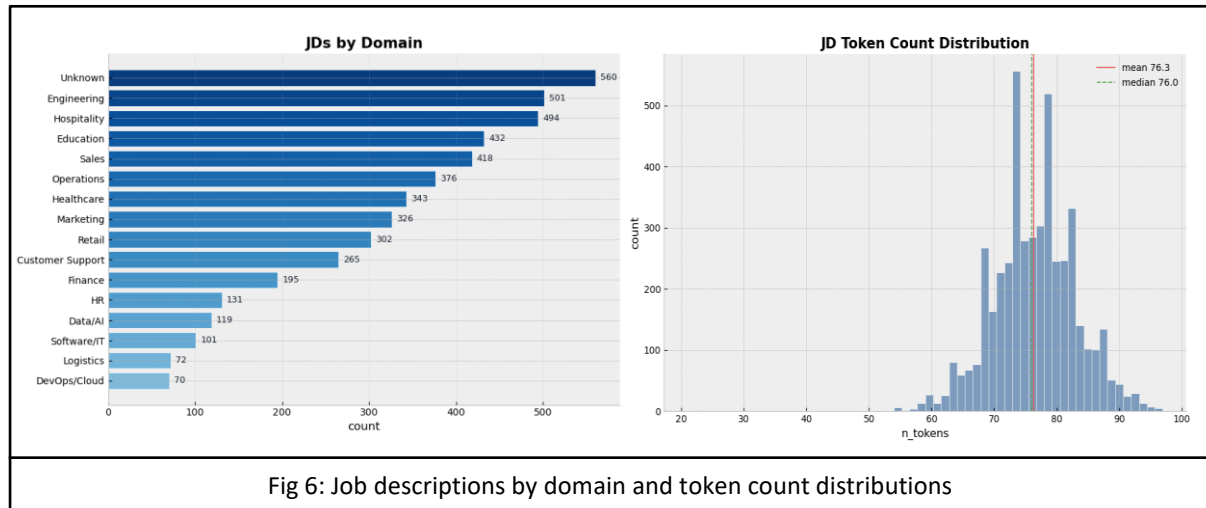


Fig 6: Job descriptions by domain and token count distributions

Overall, from these figures, it can be seen that both the datasets indicate that the corpora are diverse and realistic but exhibit domain and length imbalances. These insights informed the need for normalization, robust text representations, and model architectures capable of generalizing effectively across varied job sectors and candidate profiles.

3.3. Text Representation

The system represents text with **TF-IDF** over the **concatenated pair** of job description and resume (formatted as “[JD]... [RESUME]...”). This sparse representation keeps features interpretable and aligns directly with a linear classifier’s weights. In addition, the app computes a simple **TF-IDF cosine similarity** between JD and resume texts and a **JD-term overlap score** that focuses on high-IDF terms from the JD.

During early experiments, dense embeddings (bi-encoders) were also prototyped, but the final deployed system uses TF-IDF exclusively due to its transparency, lighter footprint, and stable performance after calibration.

3.4. Resume-JD Scoring and Calibration

The scoring technique proceeds in two steps:

- (i) The concatenated TF-IDF pair is fed to a **Logistic Regression classifier** to produce a raw log-odds score, which is then converted to a calibrated probability using **isotonic (or Platt/sigmoid) calibration** trained on a held-out validation set.
- (ii) The Streamlit application computes a fused score that blends calibrated probability with surface similarity signals:

fused = $\alpha p_{\text{cal}} + \beta \cos(\text{TF-IDF}(\text{JD}), \text{TF-IDF}(\text{Resume})) + \gamma \text{JD_term_overlap}$
 with $\alpha + \beta + \gamma = 1$ (defaults $\alpha = 0.60$, $\beta = 0.30$, $\gamma = 0.10$). A small down-weight is applied if high-value JD terms are missing from the resume to discourage superficial matches. Decisions use a fixed **Pass** or **Review** threshold on the

calibrated probability chosen from the validation Precision-Recall curve (shortlisting / high-precision operating point).

3.5. Explainability and Feedback Generation

Explanations are derived directly from the linear model. For any scored pair, the system computes **per-term contributions** as **feature value x model weight**. This surfaces the **top positive (helpful)** and **top negative (hurting)** terms. A waterfall chart shows the cumulative path from baseline (intercept) to the final log-odds by adding these contributions in magnitude order. It is a SHAP-like waterfall presentation for a linear model.

Separately, a JD-term presence view lists the most important JD terms which have **high-IDF and positive-weight** and marks which ones are covered in the resume and which ones are missing. This yield an intuitive coverage percentage.

Together, these views explain both why the score is what it is and what to change (e.g., strengthen relevant terms that matter for this JD, reduce off-target wording) without black-box attention maps.

3.6. Model Selection

Different types of models were trained, evaluated and compared. The models were:

- TF-IDF cosine baselines
- A Calibrated Logistic Regression on TF-IDF pair features
- A Fused Scorer (probability + cosine + overlap)
- Early Bi-Encoder prototypes.

Evaluation metrics used were MRR@K, Recall@{5, 10, 50} for retrieval-style ranking and precision / recall / F1 on held-out pairs, plus PR curves with marked operating points and reliability curves to verify calibration.

The calibrated Logistic Regression + fusion delivered the best balance: strong ranking metrics, well-behaved probabilities, and clear term-level explanations. Bi-encoders were not retained due to heavier inference cost and reduced interpretability in the final app.

3.6.1. Alternative classifiers and rationale for Logistic Regression

During model selection, several families beyond Logistic Regression were considered.

- Linear SVM / SGD classifiers achieve similar accuracy on sparse TF-IDF and train quickly, but their margins are not probabilistic and therefore require an additional calibration step. The resulting explanations are still weight-based but the decision scores are less directly interpretable.
- Naïve-Bayes offers extremely fast baselines, yet its conditional independence assumption typically leads to weaker ranking and poorly calibrated probabilities on real resumes and JDs.

- Tree-based ensembles (Random Forests, Gradient Boosting / XGBoost) can model non-linearity but are not naturally suited to very high-dimensional, sparse text. They tend to grow large, infer more slowly, and produce explanations that are less cleanly attributable to specific terms, while still needing probability calibration.
- Transformer encoders (Bi-encoders or Cross-Encoders) capture semantic similarity well and can improve retrieval when used as re-rankers but they are substantially heavier, harder to keep well-calibrated, and less transparent at the term level for an interactive app.

Given the project requirements, the Logistic Regression on TF-IDF pair features was adopted as the primary classifier. It provides a direct mapping from model weights to words and phrases, admits reliable isotonic / Platt calibration, and enables simple, faithful explanations and JD-term presence analyses without sacrificing responsiveness.

3.7. System Integration

The application is implemented in **Streamlit**. Users upload a PDF resume or paste text and provide a JD (paste or public URL). Text is extracted (PyPDF2 for PDFs; HTML parsed with BeautifulSoup), normalized by the same cleaning pipeline as training, and transformed with the saved TF-IDF vectorizer. The app loads the trained Logistic Regression and calibration artifacts (Joblib), computes the calibrated probability and fused score, and renders:

- (i) **Top positive / negative terms**
- (ii) **A waterfall from baseline to prediction**
- (iii) **A JD-term presence panel with coverage**

A fixed shortlisting threshold from the validation PR curve produces a clear Pass / Review decision.

This methodology delivers an explainable Resume-JD matcher that couples calibrated linear modeling with transparent term-level evidence. TF-IDF pair features keep the system lightweight and auditable, while the fused score adds robustness across domains. The Streamlit app turns scores into actionable guidance showing what helped, what hurt and which JD terms to address, supporting both fairer screening and practical resume improvement.

4. RESULTS & EVALUATION

4.1. Experimental Setup & Baselines

The evaluation was done on Adzuna Gold Pairs (JD-Resume, label $\in \{\text{Positive (1), Negative (0)}\}$) derived from the Adzuna CSV. Splits are by JD to prevent leakage: all pairs for a JD remain within a single fold (train/validation/test). For each JD a candidate pool consisting of the gold positive(s) and sampled non-matches were created and every resume was scored against the JD. The validation split is used to select the fixed pass threshold and to fit probability calibration (isotonic/ Platt). The test split is held out for final reporting. Unless

noted, features are TF-IDF over the concatenated pair (“[JD] ... [RESUME] ...”), classifier is Logistic Regression (L2), and probabilities are calibrated. The app’s headline score uses the fused signal:

$$\text{fused} = \alpha p_{\text{cal}} + \beta \cos(\text{TF-IDF}(\text{JD}), \text{TF-IDF}(\text{Resume})) + \gamma \text{JD_term_overlap}$$

$$\alpha + \beta + \gamma = 1 \text{ and defaults } \alpha = 0.60, \beta = 0.30, \gamma = 0.10$$

The baselines used were:

- **TF-IDF cosine only (no learning):** rank resumes for each JD by cosine similarity.
- **BIENC:** Sentence-BERT bi-encoder cosine similarity.
- **LR (pair) + calibration:** Logistic Regression on [air TF-IDF with isotonic/Platt calibration (the learning baseline).
- **Fused:** Calibrated LR blended with cosine and JD-term overlap.
- **RERANK:** A very shallow term-overlap re-ranker (included as a weak baseline)

The following metrics were reported:

- **MRR@5** (Mean Reciprocal Rank within top-5)
- **Recall@k** for $k \in \{5, 10, 50\}$
- Precision-Recall with Average Precision (AP) and chosen operating thresholds
- **NDCG@k** to capture both coverage and position-weighted ranking quality

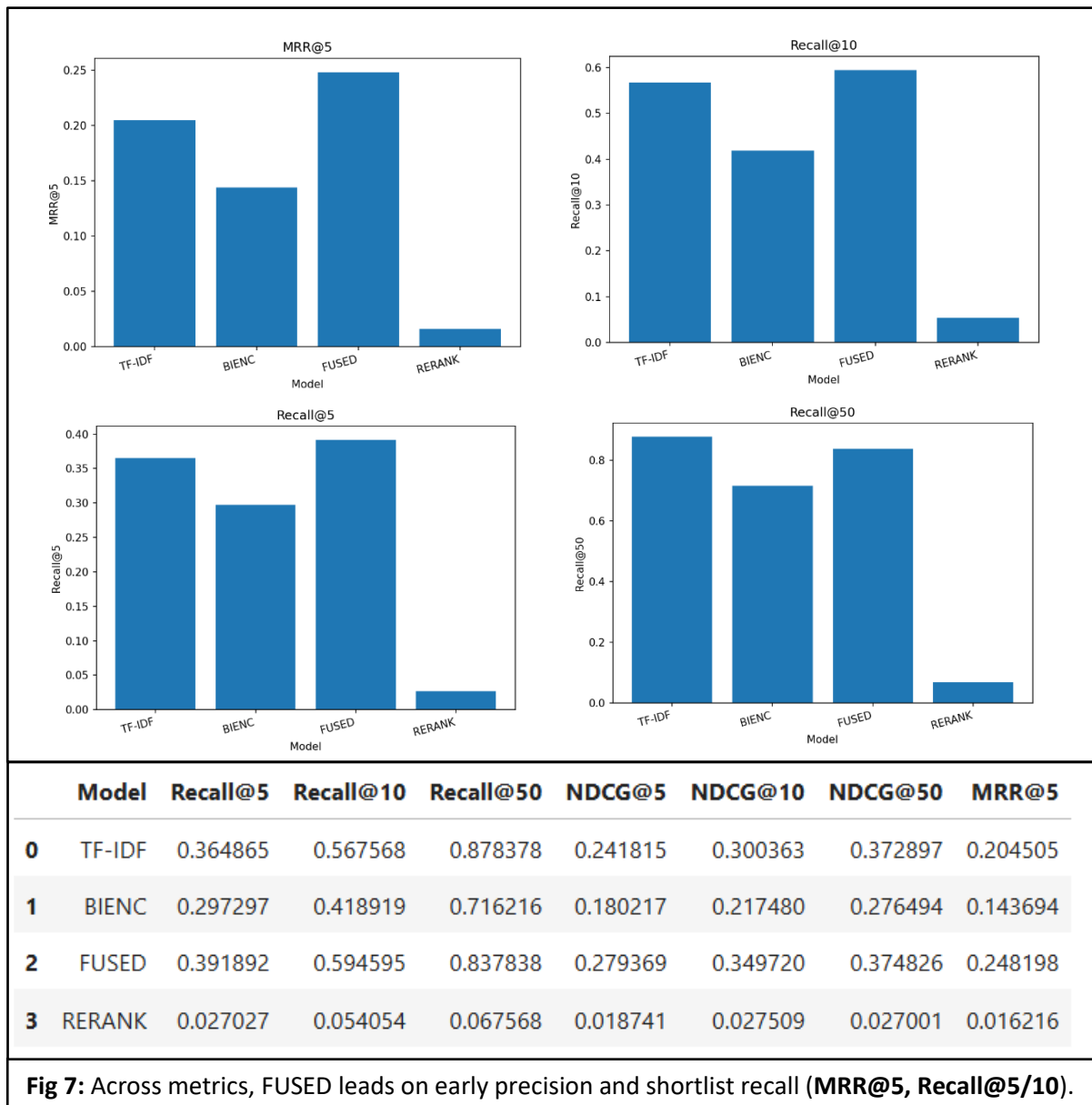
4.2. Evaluation

4.2.1 Ranking Results (Retrieval-style)

For each job description, all resumes were ranked and **MRR@5**, **Recall@{5, 20, 50}** were computed. Figure 7a – 7d show the bar charts. The Fused model consistently outperforms TF-IDF cosine and LR (pair) uncalibrated), indicating that calibrated confidence along with the surface similarity reduces both false positives and off-domain matches.

From the bar charts and the table below, it can be inferred that:

- The **FUSED** model score consistently gives the **best early precision (highest MRR@5: 0.248 and Recall@5/10: 0.39/0.59)**, which is what matters for shortlisting. It also leads NDCG across k, indicating the best top-rank ordering.
- Plain **TF-IDF** retains the strongest long-tail recall (**Recall@50: 0.878**), which indicates that lexical similarity alone can still surface positives deeper in the list, but it is weaker at ranking the very top.
- **BIENC** underperforms here likely due to domain drift and lack of in-domain fine-tuning.
- **RERANK** behaves as expected for a sanity-check baseline.

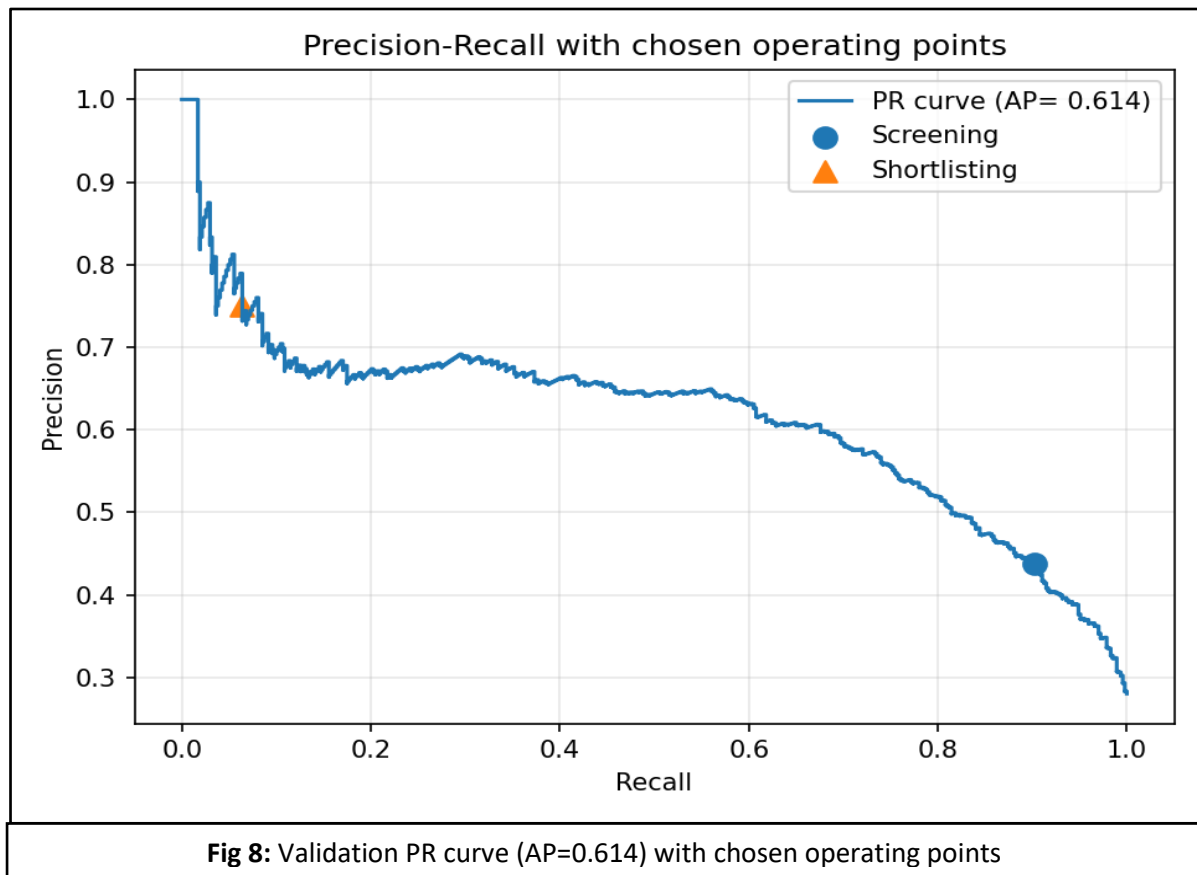


4.2.2. Precision–Recall & operating points

Figure 8 shows the validation Precision-Recall (PR) curve for the pairwise Logistic Regression model. It can be seen that Average Precision $AP = 0.614$. Two operating points are marked:

- **Screening (recall-first):** high recall (~ 0.9) at moderate precision (~ 0.4) for exploratory use.
- **Shortlisting (precision-first):** Precision ~ 0.75 at low recall ($\sim 0.06 - 0.08$).

The app's fixed threshold (τ_{pass}) is set to 0.70 due to the shortlisting point to prioritize precision and keep false positives low. The PR curve motivates one strict operating threshold for the app, with a looser point retained for analysis.



4.2.3. Probability Calibration (Reliability)

Good thresholds need well-calibrated probabilities. The curves in figure 9 plot accuracy vs. predicted confidence (reliability diagrams) using 10 equal-width bins.

- Raw Logits $\rightarrow$ Probability (uncalibrated, top-left curve): Noticeably over-confident in the 0.7 - 0.8 region (points above the diagonal), meaning scores look better than they should.
- Platt / Sigmoid Calibration (top-right curve): Improves monotonicity and brings mid-range bins closer to the diagonal, but still underfits the extremes.
- Isotonic calibration (bottom curve): Closest to the identity line overall, especially in low-mid confidence bins, yielding the most trustworthy probabilities.

Therefore, the isotonic calibration is used in the app. With Isotonic-calibrated probabilities, the fixed τ_{pass} chosen on the PR curve behaves predictably across JDs. Pass rates align with the intended precision level, and used can interpret the probability scale sensibly.

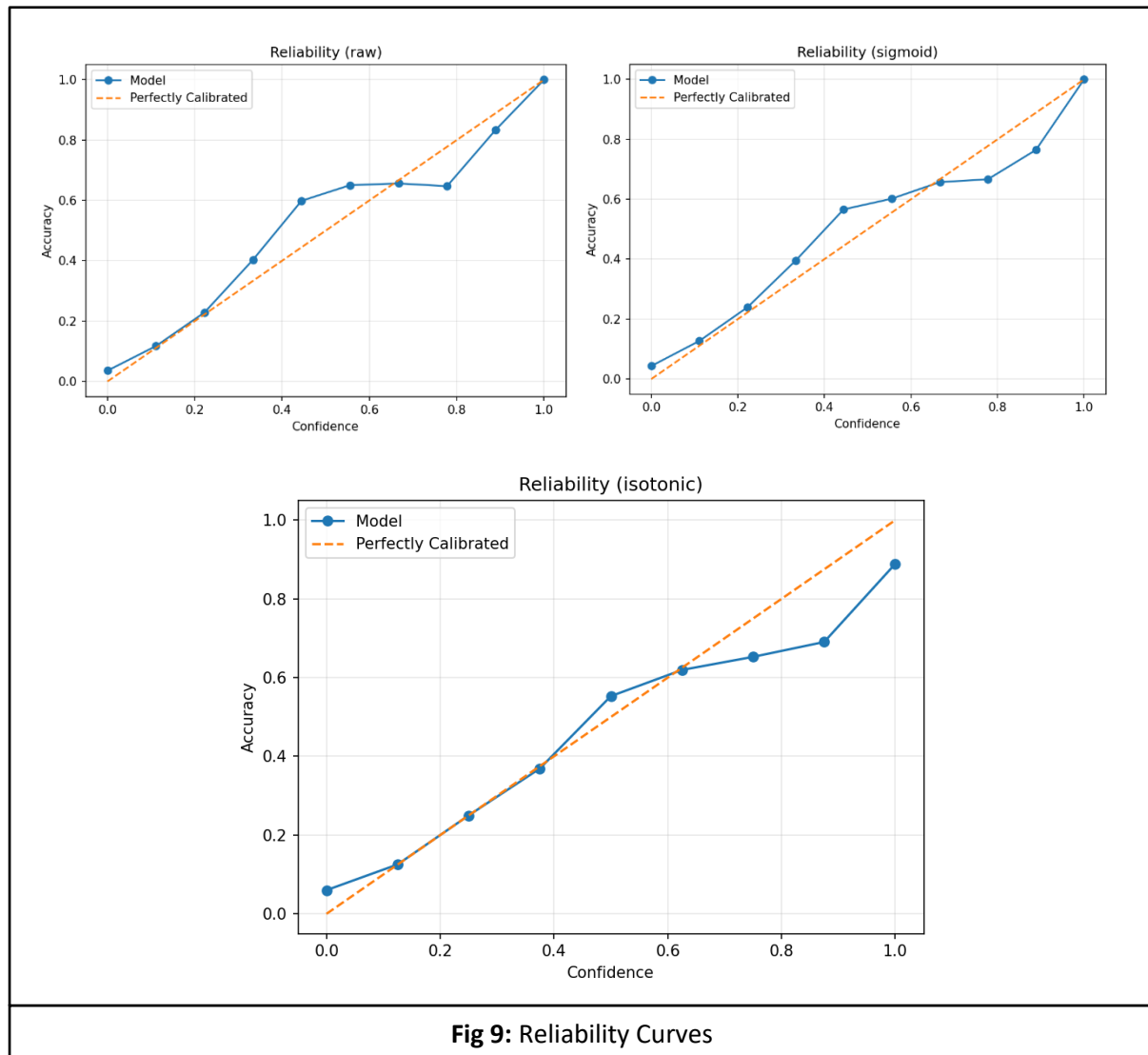


Fig 9: Reliability Curves

4.3. Explainability, Fidelity & Robustness

4.3.1. Faithfulness of Explanations (Surrogate Fidelity)

To check that the term-level explanations reflect the classifier's behavior, a sparse **Lasso surrogate** was trained on TF-IDF features to predict the calibrated probability on the held-out test pairs. The surrogate explained $\sim 70\%$ of the variance in the model outputs ($R^2 = 0.696$), which indicates **good faithfulness** for a linear, token based model (Figure 10).

4.3.2. Counterfactual Sanity Check

Another test was done whether adding a single skill token yields a sensible, small change in confidence (guarding against "magic words"). Adding a "python" to a real test example increased the calibrated probability from 0.1997 to 0.2001 (change = +0.00044). It is marked as a positive change but tiny in magnitude. This suggests the model does not over-react to a

single keyword and requires broader context to move decisions, which is consistent with our pair-wise TF-IDF and calibration design (Figure 11).

```
sur_vec = TfidfVectorizer(min_df=1).fit(pairs_te["text"].astype(str))
X_sur = sur_vec.transform(pairs_te["text"].astype(str))
y_sur = proba_te
sur = Lasso(alpha=1e-3).fit(X_sur, y_sur)
pred_sur = sur.predict(X_sur)

y_mean = np.mean(y_sur)
ss_tot = np.sum((y_sur - y_mean) ** 2) + 1e-8
ss_res = np.sum((y_sur - pred_sur) ** 2)
fidelity_r2 = float(1 - ss_res / ss_tot)

fidelity_r2

0.6960164006414746
```

Fig 10: Surrogate Fidelity

```
def counterfactual_validity_example(clf, vec, text, feature_to_add="python"):
    X0 = vec.transform([text])
    p0 = clf.predict_proba(X0)[:, 1][0]
    if feature_to_add.lower() in text.lower():
        return {"p_before": float(p0), "p_after": float(p0), "delta": 0.0, "changed": False}

    text2 = text + " " + feature_to_add
    X1 = vec.transform([text2])
    p1 = clf.predict_proba(X1)[:, 1][0]
    return {"p_before": float(p0), "p_after": float(p1), "delta": float(p1 - p0), "changed": True}

example_text = pairs_te.iloc[0]["text"]
cf_result = counterfactual_validity_example(clf, vec, example_text, feature_to_add="python")
cf_result

{'p_before': 0.19967442516326914,
 'p_after': 0.20011196240166199,
 'delta': 0.00043753723839284886,
 'changed': True}
```

Fig 11: Counterfactual Sanity Check

4.3.3. Robustness under domain shift & keyword stuffing

Performance was evaluated by JD category on the test split and ran an adversarial keyword-stuffing probe. It can be seen that performance is consistently good in several domains (**e.g., Trade & Construction Jobs**), while IT is lower, likely due to denser jargon and closer negatives. This argues for per-domain threshold tuning (or light re-calibration) if the system is deployed broadly.

For Adversarial keyword-stuffing, “python” was appended to the same test example 20 times. The probability moved from 0.1997 to 0.2035 (change = +0.0038), a < **0.4 percentage-point**

```

pairs_te = pairs_te.copy()
pairs_te["category"] = pairs_te["job_title"].astype(str).map(cat_lookup).fillna("Unknown")

def metric_on_subset(sub):
    if len(sub) == 0:
        return {"roc_auc": None, "f1": None}
    Xs = vec.transform(sub["text"].astype(str).tolist())
    ys = sub["label"].astype(int).values
    ps = clf.predict_proba(Xs)[: , 1]
    pr = (ps >= 0.5).astype(int)
    return {"roc_auc": float(roc_auc_score(ys, ps)), "f1": float(f1_score(ys, pr))}

cats = pairs_te["category"].unique().tolist()
robustness_report = {c: metric_on_subset(pairs_te[pairs_te["category"] == c]) for c in cats}

def score_text(t):
    return float(clf.predict_proba(vec.transform([t]))[: , 1][0])
before = score_text(example_text)
after = score_text(example_text + " " + (" python" * 20))
robustness_report["adv_keyword"] = {"before": float(before), "after": float(after), "delta":
robustness_report

{
  'Hospitality & Catering Jobs': {'roc_auc': 0.8623188405797102,
    'f1': 0.5365853658536586},
  'IT Jobs': {'roc_auc': 0.7024339504888704, 'f1': 0.34355828220858897},
  'Trade & Construction Jobs': {'roc_auc': 0.9058823529411764,
    'f1': 0.7619047619047619},
  'Engineering Jobs': {'roc_auc': 0.8437279774489076, 'f1': 0.4126984126984127},
  'PR, Advertising & Marketing Jobs': {'roc_auc': 0.8533971728226173,
    'f1': 0.6575342465753424},
  'adv_keyword': {'before': 0.19967442516326914,
    'after': 0.20346931823031464,
    'delta': 0.0037948930670455017}}

```

Fig 11: Counterfactual Sanity Check

change. This small effect indicated the model is **robust to naïve keyword stuffing**, which is desirable for a candidate-facing tool.

4.4. App UI & Explainability

A user can upload a resume or paste the text of a resume, on the right hand side, there are options to paste the text of a JD or provide a link to the JD. Then the user can click on the “Score my resume” button and get the score and feedback.

4.4.1. Overview Badges (Figure 12)

- **Match Score (0-100)** is the fused signal: calibrated LR probability + TF-IDF cosine + JD-term overlap.
- **Decision (Pass/Review)** uses the fixed **shortlisting** threshold chosen on the validation PR curve; an optional **screening** (looser) threshold is also shown in the app.

- **Calibrated probability** is post-processed (isotonic / Platt), so a value of $p = 0.80$ means ~80% of similar cases were true positives on validation.
- **JD term coverage** reports presence of the most predictive JD terms. Low coverage down-weights the fused score.

Your resume & the job description

Upload your resume (PDF)

Drag and drop file here
Limit 200MB per file • PDF

Browse files

Jordan Smith.pdf 5.0KB

Or paste your resume text

Jordan Smith Email: jordan.smith@datamail.com | Phone: +1 (555) 246-8102 LinkedIn: linkedin.com/in/jordansmith-ds | GitHub: github.com/jordansmith-ds Professional Summary Data Scientist with extensive experience in machine learning, predictive analytics, natural language processing, and big data solutions. Proficient in Python, R, SQL, and cloud platforms (AWS, Azure, GCP). Expert in designing predictive models, building ETL pipelines, and deploying scalable AI/ML systems. Strong background in MLOps, A/B testing, and statistical modeling for business optimization. Technical Skills Programming: Python, R, SQL, Scala, Spark Machine Learning: Regression, Classification, Clustering, NLP, Deep Learning, Recommendation Systems

How will you provide the job description?

☐ Paste text ☒ Paste URL

Paste the JD link (public page)

<https://www.datascientistjobs.co.uk/job/data-scientist-public-sector-2>

Job description text

A career in IBM Software means you'll be part of a team that transforms our customer's challenges into solutions. Seeking new possibilities and always staying curious, we are a team dedicated to creating the world's leading AI-powered, cloud-native software solutions for our customers. Our renowned legacy creates endless global opportunities for our IBMers, so the door is always open for those who want to grow their career. IBM's product and technology landscape includes Research, Software, and Infrastructure. Entering this domain positions you at the heart of IBM, where growth and innovation thrive In this role, you'll work in one of our IBM Consulting Client Innovation Centers (Delivery Centers), where we deliver deep technical and

Match score	Decision	Calibrated probability	JD term coverage (top)
69 / 100	☒ Pass	1.00	100%

Fig 12: App UI

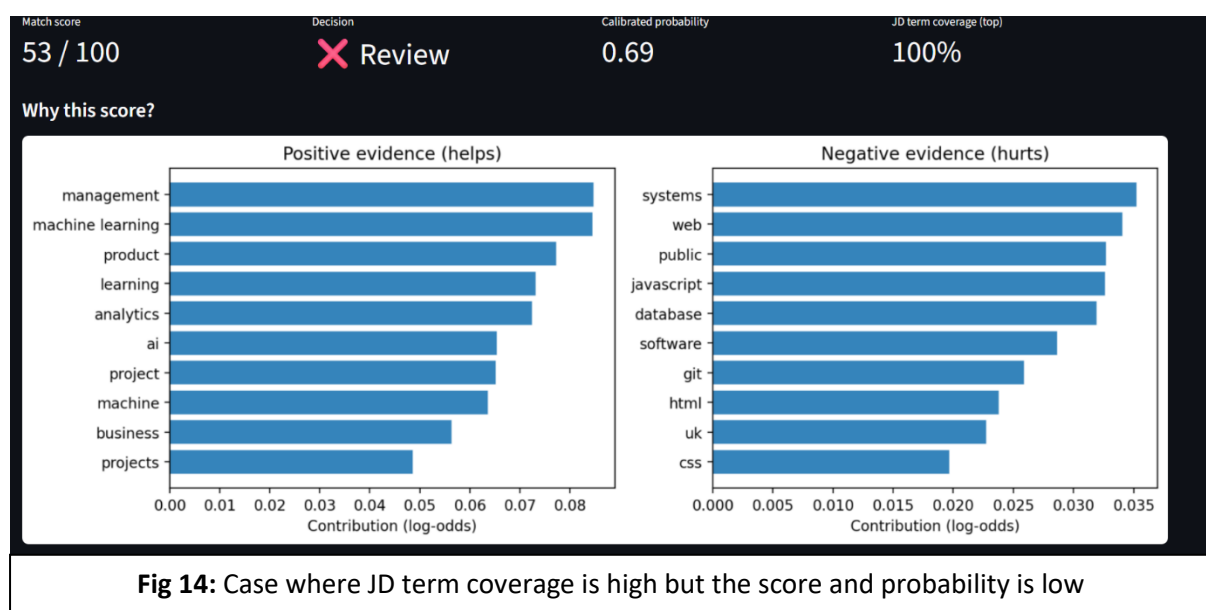
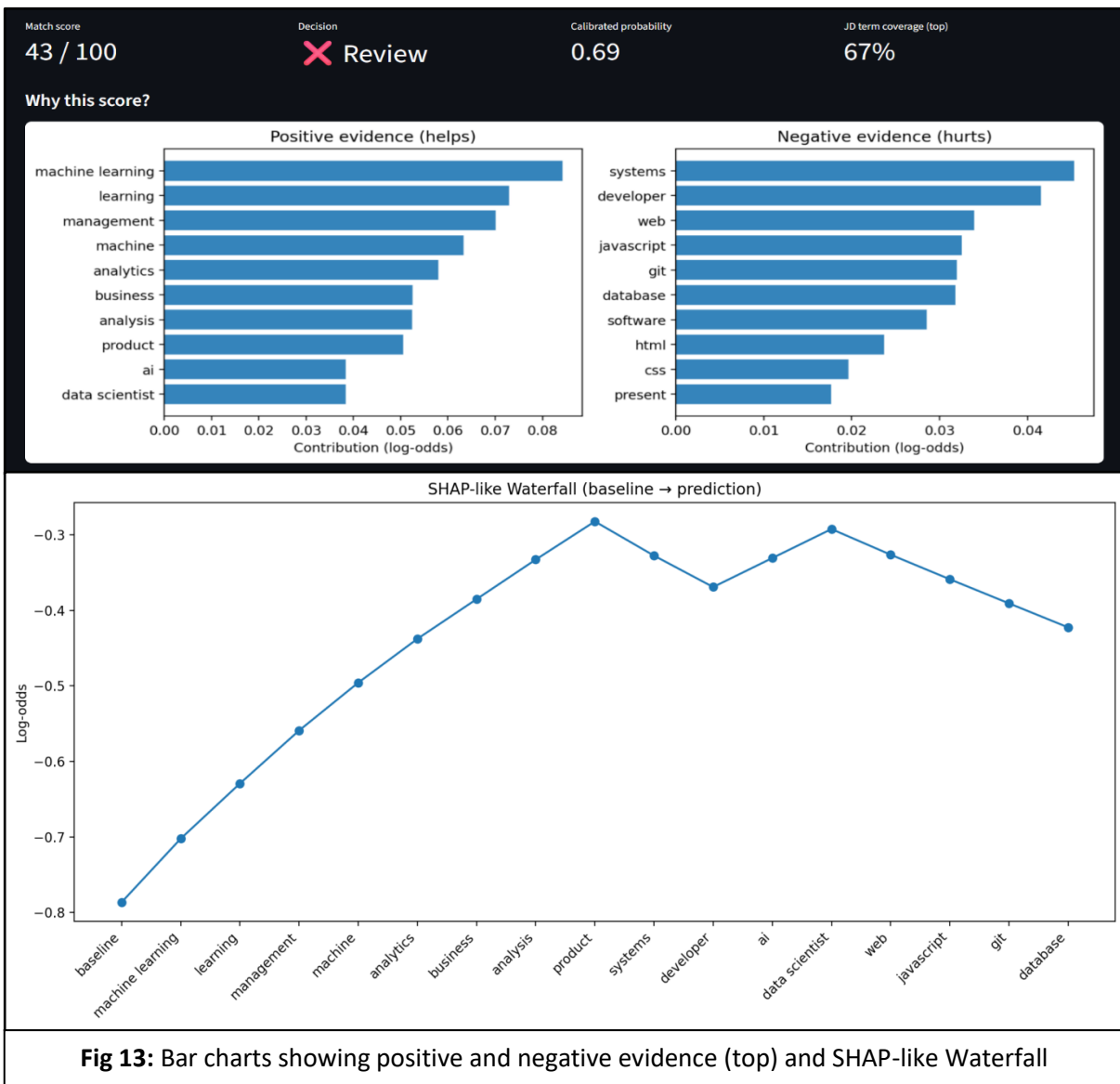
4.4.2. “Why this score?” bars & SHAP-like Waterfall

After the user clicks on the button, along with the score, decision, calibrated probability and JD term coverage, two bar charts and a waterfall chart are presented. The two bar charts (figure 13, top) show **term-level contributions (log-odds)** from the linear model:

- **Positive Evidence (helps)** – terms that pushed the score up
- **Negative Evidence (hurts)** – terms that pushed the score down

These are directly derived from the model weights x TF-IDF values. Therefore, they are faithful and not post-hoc approximations. A waterfall chart gives a left-to-right path from the model intercept (**baseline**) to the final log-odds. Each step shows how a high-impact term **raises** or **lowers** the prediction. For example, in figure 13 bottom-left, the waterfall diagram can be inferred as such: “after **machine learning** the score rose, after **web/javascript** it dropped,” etc. The figure on the bottom right shows a list of top JD terms present in the resume in the left column and the right column lists to missing ones.

After the user or candidate receives the feedback, they can take actions such as if the **coverage is low**, they can add concrete, truthful evidence for the missing JD terms. If **coverage is high but probability is low**, they can **neutralize hurt tokens** (e.g., rephrase “JavaScript / HTML / CSS web apps” → “built lightweight internal dashboards in Streamlit / Dash” [figure 14]) and mirror the JD’s phrasing in your summary to increase alignment. Also, putting the relevant terms like MLOps, ML outcomes and pushing the generic terms like software to the bottom can increase the score.



4.5. Error Analysis (What goes wrong and Why)

Most misses fell into four buckets:

- (1) Vocabulary mismatch / paraphrase: The JD says “client-facing delivery in cloud-native platforms”, while the resume uses different phrasing. In this case TF-IDF can underweight these, lowering calibrated p despite good coverage.
- (2) Off-target tokens: Generic “developer / web / systems / HTML / CSS” cues carry negative weights for DS roles and can drag scores down even when the right skills are present. The waterfall plots make these penalties visible
- (3) JD quality and domain shift: Vague or sales-heavy JDs, or niche sub-domains, produce weak signals. Per-category robustness showed variance in ROC-AUC and F1 across sectors, and extreme-length resumes occasionally distort TF-IDF.
- (4) Input noise: Bad OCR, duplicated bullet boilerplate, and keyword stuffing (e.g., repeating “python”) had limited but non-zero impact. Counterfactual and “stuffing” checks show only small score changes but they explain occasional false positives or negatives. Overall fidelity of the linear surrogate is high enough to trust the per-term attributions, and calibration plots indicate probabilities are reliable except at the very extremes.

5. DISCUSSIONS & LIMITATIONS

5.1. Discussion

During the evaluation, it is inferred that combining calibrated model confidence with surface-level similarity produces better early rank quality than either signal alone. The fused score, calibrated Logistic Regression on pairwise TF-IDF blended with TF-IDF cosine and JD-term overlap consistently lifts MRR@5 and Recall@5 /10, which are the metrics that matter for shortlisting and user-perceived relevance. Plain TF-IDF remains competitive at very deep cutoffs (Recall@50), which is expected from a lexical method that sweeps broadly, while a generic bi-encoder underperforms without in-domain fine-tuning. The Precision-Recall analysis justifies a fixed, high-precision pass threshold for the app and the reliability plots confirm that isotonic or Platt calibration makes probabilities behave sensibly. So a given cutoff translates into stable acceptance rates across JDs.

The explainability views help turn a binary decision into actionable guidance. Term-level contributions expose which words and phrases pushed the score up or down, the JD-presence view quantifies how much of the important JD signal is actually present in the resume, and the waterfall chart offers a compact, baseline-to-prediction narrative. In practice, we observe a common “coverage-confidence gap”: candidates may mention most of the JD’s important terms yet still score modestly because off-domain cues contribute negatively. When those are truthfully reframed or de-emphasized, calibrated probability and the fused score rise in a way that is visible to the user and auditable for supervision. Overall, the approach balances three goals: ranking quality, calibration for a fixed operation point, and human-understandable reasons, while remaining fast enough for an interactive app.

Compared with typical “black-box” matchers (e.g., BERT/SBERT bi-encoders or commercial semantic rankers), the Explainable AI-Based Resume Scorer trades a little raw semantic capacity for calibrated, auditable decisions and term-level reasons. Deep encoders are strong at paraphrase, but they usually return opaque scores and need extra plumbing to be well-calibrated. The pipeline presented is deliberately transparent: **TF-IDF pair features + Logistic Regression yield exact per-term contributions, a JD-presence view (what’s missing)**, and a waterfall from baseline → prediction. Hence, users see *why* the decision was made and how to improve. Additionally, a fixed operating threshold from the validation PR curve was set and verified it with reliability plots, which most published matchers and LLM add-ons don’t surface. In short, it is not just ranking quality (FUSED > cosine/SBERT on MRR@5/Recall@k) but decision accountability with metrics that back it up.

Compared with **keyword-count ATS scorers and proprietary resume “fit” scores**, this system is both harder to game and more actionable. Instead of boosting scores by dumping keywords, calibrated probability + cosine + JD-term overlap is blended and showed that naive stuffing barely moves confidence (stress test), while off-target tokens are explicitly flagged as “hurts.” Candidates get targeted, truthful edits (“add evidence for these JD terms; rephrase or downplay these negatives”) rather than a mystery number. Operationally, the model is lightweight and fast (great for an interactive app), trained/evaluated on manually labeled gold pairs with JD-wise splits to prevent leakage, and ships with robustness checks (category slices, counterfactual probes). That combination of transparent features, calibrated thresholding, faithful explanations, and demonstrated robustness is precisely what many existing systems lack, making yours easier to trust, adopt, and govern.

5.2. Limitations & Future Work

The prototype presented is intentionally lightweight and candidate-facing, and that scope bring several limitations. The training and evaluation rely on Adzuna-derived gold pairs and sampled non-matches. While this is pragmatic, label noise and pool construction can bias absolute metrics and may not reflect every hiring scenario. The model uses only text extracted from PDFs and scraped JDs. The quality of the extraction varies across documents, and errors or formatting artifacts can influence both scores and explanations. Although personally identifiable information (PII) is removed and no demographic attributes are used, proxy features can still leak into text, so any fairness statements remain diagnostic rather than definitive without trusted group labels and a broader audit.

Methodologically, a linear classifier on sparse TF-IDF features was chosen to guarantee transparent term-level attributions and fast inference. That design also constrains what the model can learn, that is, linear decision boundaries may underfit some semantic nuances, and the system does not include a heavy cross-encoder re-ranker that could capture fine-grained context. A Bigram vectorizer was intentionally not deployed in the app, even though experiments show that adding phrases like “machine learning” or “project management” can yield a modest MRR@5 gain on phrase-heavy JDs. This is because the trade-offs were unfavourable for an interactive tool. A (1, 2) vectorizer inflates the vocabulary and sparsity,

increases latency and artifact size, reduces calibration stability on rare features, and make explanations noisier by surfacing many overlapping phrases. If future deployments target batch scoring or have looser latency constraints, retraining with selected phrases or a curated phrase whitelist would be a sensible upgrade.

The app currently applies a single fixed pass threshold chosen on the validation PR curve. This improves auditability but may be conservative for some roles and aggressive for others. Adaptive, JD-specific thresholds or cost-sensitive objectives could better reflect different error tolerances. Explanations are faithful for the linear model that is used, but the SHAP-like waterfall is a visualization of linear contributions than full SHAP token heatmaps. Extending to model-agnostic explanations would add computational cost and complexity. Finally, since this version focuses on the Job seeker side, there is no end-to-end evaluation of recruiter workflow impact or inter-annotator agreement on the usefulness of explanations. A natural next step is a small human study measuring time-to-edit, change in score after revision, and perceived clarity of the reasons.

6. Conclusion

Artificial Intelligence (AI) is now extensively applied in recruitment to process and evaluate large volumes of resumes, significantly reducing the workload of hiring managers. However, most of these systems operate as black-box models that offer little to no explanation for their decisions. This lack of transparency often results in mistrust, confusion, and dissatisfaction among both job seekers and recruiters. Furthermore, candidates are unable to understand why their resumes are not shortlisted for specific roles, as current systems provide analytical results but fail to deliver meaningful interpretations.

The “Explainable AI-based Resume Scoring System” delivers a working system for the job seekers that scores a resume against a job description and crucially explains the reason. Using transparent TF-IDF pair features with a calibrated Logistic Regression classifier, a fused score (probability + cosine + JD-term overlap) to a single 0-100 score, and operating points chosen on a validation PR curve, the app turns screening into an interpretable, auditable process. The interface surfaces term-level “helps vs hurts”, JD-term coverage, and a waterfall path from baseline to prediction, giving applicants concrete, truthful edits rather than opaque verdicts.

Beyond accuracy, the system’s value lies in actionability and accountability. In offline tests, the **FUSED** method consistently outperformed TF-IDF cosine and SBERT bi-encoder baselines on ranking metrics (MRR@5, Recall@5/10/50, NDCG), and calibration plots showed probabilities track observed outcomes, supporting a fixed shortlisting threshold. Robustness checks indicate sensible behavior where small changes for naïve keyword stuffing and stable performance across sectors with expected variance for niche roles.

There are, however, limits to this project. Sparse linear models miss some paraphrases, scores depend on JD quality, the labeled pairs are modest, and unigrams were kept for speed and

clarity in the app as bigrams help a little on phrase-heavy JDs at extra cost. The current system is English-centric and role-distribution may reflect our data sources. Broader, regularly-audited datasets are needed for stronger fairness guarantees.

Looking ahead, several improvements are natural such as:

- (1) Growing and diversifying gold pairs and run per-role calibration
- (2) Adding section-aware features and a lightweight cross-encoder re-ranker while retaining linear explanations
- (3) Supporting multilingual JDs and CVs and skill ontologies
- (4) Shipping an in-app “edit simulator” to preview score changes
- (5) Conducting human-in-the-loop A/B tests to measure real uplift in interview rates

Overall, the system shows that resume screening can be both **effective** and **explainable**, providing applicants actionable guidance and giving stakeholders an auditable, thresholded decision.

References

1. Adzuna. (2025). *Adzuna API documentation*. Available at: <https://developer.adzuna.com/> [Accessed 12 Sept. 2025].
2. Barocas, S. and Selbst, A.D. (2016). Big data's disparate impact. *California Law Review*, 104(3), pp.671–732.
3. Blessing, S. (2025). *AI-powered resume screening: Benefits and challenges*. ResearchGate. Available at: https://www.researchgate.net/publication/388688179_AI-Powered_Resume_Screening_Benefits_and_Challenges [Accessed 29 Sept. 2025].
4. Buolamwini, J. and Gebru, T. (2018). Gender shades: Intersectional accuracy disparities in commercial gender classification. *Proceedings of Machine Learning Research (PMLR)*, 81, pp.77–91.
5. Dastin, J. (2018). *Amazon scraps secret AI recruiting tool that showed bias against women*. Reuters. Available at: <https://www.reuters.com/article/us-amazon-com-jobs-automation-insight-idUSKCN1MK08G> [Accessed 29 Sept. 2025].
6. Fabeyo, O., Abiola, A. and Usman, M. (2025). Explainable AI in employment decision-making: A systematic review. *Issues in Information Systems*, 26(1), pp.127–135. Available at: https://iacis.org/iis/2025/3_iis_2025_127-135.pdf [Accessed 2 Sept. 2025].
7. Fabris, A., Messina, A., Vianello, A. and Pelillo, M. (2023). Fairness and bias in algorithmic hiring: A multidisciplinary survey. *arXiv preprint*. Available at: <https://arxiv.org/abs/2309.13933> [Accessed 3 Sept. 2025].
8. Google Cloud. (2025). *Cloud Talent Solution API | Job Search documentation*. Google. Available at: <https://cloud.google.com/talent-solution/job-search/docs/reference/rest> [Accessed 13 Sept. 2025].
9. Hofeditz, L., Schwanecke, U., Ziegler, J., Keller, B. and Nitsch, V. (2022). Applying XAI to an AI-based system for candidate recommendation. *Frontiers in Artificial Intelligence*, 5, p.931144. Available at: <https://pmc.ncbi.nlm.nih.gov/articles/PMC9764302/> [Accessed 3 Sept. 2025].
10. InferencePrince555. (2021). *Resume Dataset* [Data set]. Hugging Face. Available at: <https://huggingface.co/datasets/InferencePrince555/Resume-Dataset> [Accessed 30 Aug. 2025].
11. Kosinski, M. (2024). What is black box artificial intelligence (AI)? *IBM*. Available at: <https://www.ibm.com/think/topics/black-box-ai> [Accessed 29 Sept. 2025].
12. Lambrecht, A. and Tucker, C. (2019). Algorithmic bias? An empirical study into apparent gender-based discrimination in the display of STEM career ads. *Management Science*, 65(7), pp.2966–2981.
13. LinkedIn. (n.d.). *LinkedIn Talent Solutions: AI-driven recruitment tools*. Available at: <https://www.linkedin.com/talent/> [Accessed 5 Aug. 2025].
14. Lundberg, S.M. and Lee, S.I. (2017). A unified approach to interpreting model predictions. *Proceedings of the 31st International Conference on Neural Information Processing Systems (NeurIPS)*, pp.4765–4774.
15. Lundberg, S.M., Nair, B., Vavilala, M.S., Horibe, M., Eisses, M.J., Adams, T., Liston, D.E., Low, D.K.W., Newman, S.F., Kim, J. and Lee, S.I. (2018). Explainable machine-learning predictions

- for the prevention of hypoxaemia during surgery. *Nature Biomedical Engineering*, 2(10), pp.749–760.
16. Magham, R. (2024). Mitigating bias in AI-driven recruitment: The role of explainable machine learning (XAI). ResearchGate. Available at: https://www.researchgate.net/publication/384802794_Mitigating_Bias_in_AI-Driven_Recruitment_The_Role_of_Explainable_Machine_Learning_XAI [Accessed 29 Sept. 2025].
 17. Mehrabi, N., Morstatter, F., Saxena, N., Lerman, K. and Galstyan, A. (2021). A survey on bias and fairness in machine learning. *ACM Computing Surveys*, 54(6), pp.1–35.
 18. Mujtaba, G. and Mahapatra, D. (2024). Fairness in AI-driven recruitment: Challenges, metrics, methods, and future directions. *arXiv preprint*. Available at: <https://arxiv.org/abs/2405.19699> [Accessed 27 Sept. 2025].
 19. Raghavan, M., Barocas, S., Kleinberg, J. and Levy, K. (2020). Mitigating bias in algorithmic hiring: Evaluating claims and practices. *Proceedings of the ACM Conference on Fairness, Accountability, and Transparency (FAT20)**, pp.469–481.
 20. Ribeiro, M.T., Singh, S. and Guestrin, C. (2016). “Why should I trust you?”: Explaining the predictions of any classifier. *Proceedings of the 22nd ACM SIGKDD International Conference on Knowledge Discovery and Data Mining (KDD)*, pp.1135–1144.
 21. University of Washington. (2024). *AI shows bias in resume screening for race and gender*. University of Washington News. Available at: <https://www.washington.edu/news/2024/10/31/ai-bias-resume-screening-race-gender/> [Accessed 28 Sept. 2025].
 22. VoxDev. (2024). *AI hiring tools exhibit complex gender and racial biases*. VoxDev. Available at: <https://voxdev.org/topic/technology-innovation/ai-hiring-tools-exhibit-complex-gender-and-racial-biases> [Accessed 29 Sept. 2025].